Python Variables — Complete Notes for Beginners

Video: Learn Python VARIABLES in 10 minutes! ❎

Video link: https://www.youtube.com/watch?v=7IoQ5BGkTJo

Topic: Variables and Basic Data Types in Python

1. What Is a Variable?

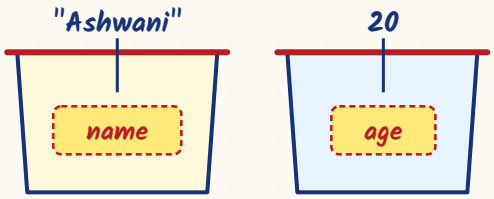
★ Most Important

A variable is a named reference used to store or represent a value in a program. It allows us to reuse and manipulate data without repeatedly writing the same value.

Think of a variable as a labelled container.

Example:

```
name = "Ashwani"
age = 20
```



Labelled Containers in Memory

Here:

- **name** is a variable that refers to the string **"Ashwani"**.
- **age** is a variable that refers to the integer **20**.

We can use these variables anywhere they are accessible in our program.

```
print(name)
print(age)
```

Output:

```
Ashwani
20
```

Important points

- Every variable has a name.
- A variable is created when a value is assigned to it.
- Variables can be reused in calculations, output statements, and conditions.
- Python is dynamically typed, so we do not need to declare a variable's data type explicitly.

2. Assignment Operator (=)

Frequently Asked

The assignment operator = assigns a value to a variable.

Syntax:

```
variable_name = value
```

Examples:

```
name = "Ashwani"
age = 20
price = 99.50
is_student = True
```

In these examples, Python assigns a value to each variable.

Assignment vs. equality

Do not confuse = with ==.

Operator	Meaning	Example
=	Assignment	age = 20
==	Equality comparison	age == 20

```
age = 20
print(age == 20)
```

Output:

```
True
```

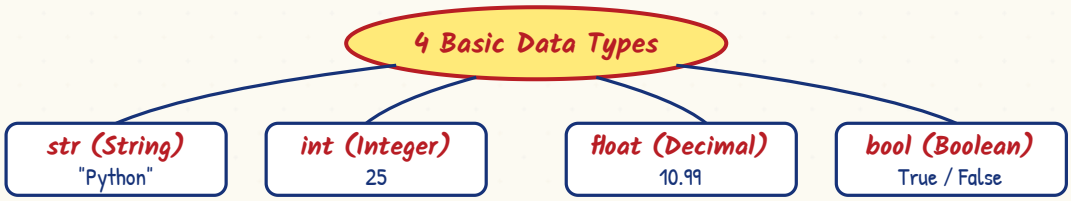
The first statement assigns `20` to `age`, while the second checks whether `age` equals `20`.

3. The Four Basic Data Types

★ Most Important

The video introduces four fundamental Python data types:

Data type	Meaning	Example
`str`	Text or a sequence of characters	`"Python"`
`int`	Whole numbers	`25`
`float`	Numbers with a fractional part	`10.99`
`bool`	Truth values	`True`, `False`



Let's explore each type in detail.

4. String (`str`)

A string is a sequence of characters, such as letters, words, symbols, or digits represented as text.

Strings are enclosed in single or double quotation marks.

```
first_name = "Ashwani"
food = "Pizza"
email = "ashwani@example.com"
```

Python treats the values inside quotation marks as strings, even when they contain digits.

```
number_text = "123"
```

Here, `number_text` is a string, not an integer.

Single quotes vs. double quotes

Both are valid in Python.

```
name = "Ashwani"
city = 'Darbhanga'
```

For these simple examples, both forms work the same way. Choose one style and use it consistently.

Displaying a string variable

```
first_name = "Ashwani"
print(first_name)
```

Output:

```
Ashwani
```

Notice that `first_name` is written without quotation marks inside `print()`.

Compare the following:

```
print(first_name)
print("first_name")
```

Output:

```
Ashwani
first_name
```

- `print(first_name)` displays the value stored in the variable.
- `print("first_name")` displays the literal text `first_name`.

Using strings with other text

Suppose we want to display a greeting that includes a variable's value.

Python provides **f-strings**, which make this easy.

Shortcut

5. F-Strings — Formatting Output

An **f-string** is a formatted string literal that lets us insert variable values directly into a string.

Syntax:

```
f"Text {variable_name}"
```

The letter `f` comes before the opening quotation mark. Variables or expressions are inserted inside curly braces `{}`.

Prefix f

```
f "Hello,  {first_name} !"
```

Curly braces evaluate variable

Example 1: Greeting

```
first_name = "Ashwani"

print(f"Hello, {first_name}!")
```

Output:

```
Hello, Ashwani!
```

Example 2: Combining two variables

```
first_name = "Ashwani"
food = "Pizza"

print(f"Hello, {first_name}. You like
{food}.")
```

Output:

```
Hello, Ashwani. You like Pizza.
```

Example 3: Displaying an email

```
email = "ashwani@example.com"

print(f"Your email is {email}")
```

Output:

```
Your email is ashwani@example.com
```

Example 4: Performing calculations inside an f-string

```
price = 100
quantity = 3

print(f"Total cost: ₹{price * quantity}")
```

Output:

```
Total cost: ₹300
```

Key takeaway: Use f-strings when you want to mix ordinary text with variable values or expressions.

6. Integer (`int`)

An **integer** is a whole number without a fractional or decimal part.

Examples include:

```
age = 20
quantity = 3
num_students = 30
temperature = -5
```

Integers can be positive, negative, or zero.

Example 1: Age

```
age = 20

print(f"You are {age} years old.")
```

Output:

```
You are 20 years old.
```

Example 2: Quantity

```
quantity = 3

print(f"You are buying {quantity} items.")
```

Output:

```
You are buying 3 items.
```

Example 3: Number of students

```
num_students = 30

print(f"Your class has {num_students} students.")
```

Output:

```
Your class has 30 students.
```

Integers support arithmetic

```
a = 10
b = 5

print(a + b)
print(a - b)
print(a * b)
print(a // b)
```

Output:

```
15
5
50
2
```

Here, `//` performs floor division. With these positive integers, it produces the whole-number quotient.

Important distinction: Integer vs. string

Frequently Asked

```
age = 20
age_text = "20"

print(type(age))
print(type(age_text))
```

Output:

```
<class 'int'>
<class 'str'>
```

Although both values look like `20`, their data types differ.

7. Float (`float`)

A `float` is a number represented with a fractional or decimal part.

Examples:

```
price = 99.50
gpa = 3.2
distance = 5.5
```

Floats are useful for representing measurements, prices, averages, and other values that can have fractions.

Example 1: Price

```
price = 99.50

print(f"The price is ₹{price}")
```

Output:

```
The price is ₹99.5
```

Python does not necessarily preserve trailing zeros when displaying a float.

To display two decimal places, use the following formatting:

```
print(f"The price is ₹{price:.2f}")
```

Output:

```
The price is ₹99.50
```

Example 2: Grade point average

```
gpa = 3.2

print(f"Your GPA is {gpa}")
```

Output:

```
Your GPA is 3.2
```

Example 3: Distance

```
distance = 5.5

print(f"You ran {distance} km.")
```

Output:

```
You ran 5.5 km.
```

Integer vs. float

Integer	Float
10	10.5
25	25.75
0	0.0

Even though **10** and **10.0** represent the same numerical value, Python treats them as different types.

```
print(type(10))
print(type(10.0))
```

Output:

```
<class 'int'>
<class 'float'>
```

Remember: A value written with a decimal point, such as **10.0**, is a float.

8. Boolean (bool)

Memory Trick: Capital T & F

A Boolean is a data type with only two possible values:

- **True**
- **False**

Booleans are commonly used to represent conditions, such as whether a student is enrolled, an item is available, or a user is online.

Important syntax rule

Python uses a capital first letter for Boolean values.

```
is_student = True
is_online = False
```

These are valid.

```
is_student = true
```

This is incorrect in ordinary Python code because **true** is not the Boolean literal. Python raises a **NameError** unless that name has been defined separately.

Example 1: Student status

```
is_student = True

print(is_student)
```

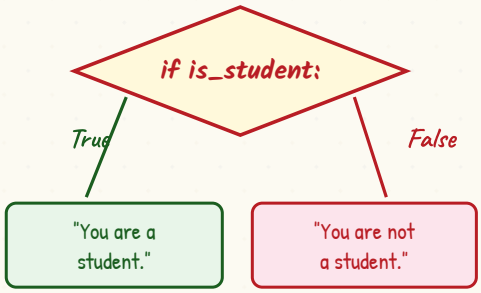
Output:

```
True
```

Example 2: Checking a Boolean using `if` and `else`

```
is_student = True

if is_student:
    print("You are a student.")
else:
    print("You are not a student.")
```



Output:

```
You are a student.
```

If **`is_student`** is changed to **`False`**, the output becomes:

```
You are not a student.
```

Example 3: Product availability

```
is_for_sale = True

if is_for_sale:
    print("The item is for sale.")
else:
    print("The item is not available for
sale.")
```

Output:

```
The item is for sale.
```

Example 4: Online status

```
is_online = False

if is_online:
    print("You are online.")
else:
    print("You are offline.")
```

Output:

```
You are offline.
```

Why are Booleans important?

Booleans help a program make decisions. They are frequently used in:

- `if` and `else` statements
- Comparisons
- Loops
- Validation and logical conditions

You can also obtain a Boolean result from a comparison:

```
age = 20

print(age >= 18)
```

Output:

```
True
```

The expression `age >= 18` evaluates to `True` because `20` is greater than or equal to `18`.

9. Choosing the Correct Data Type

Consider a college application that stores a student's information.

```
name = "Ashwani"
age = 20
percentage = 75.6
is_enrolled = True
```

Variable	Value	Data type	Reason
`name`	`"Ashwani"`	String	Represents text
`age`	`20`	Integer	Represents a whole number
`percentage`	`75.6`	Float	Represents a decimal value
`is_enrolled`	`True`	Boolean	Represents a true/false condition

To check a variable's type, use Python's built-in `type()` function.

```
print(type(name))
print(type(age))
print(type(percentage))
print(type(is_enrolled))
```

Output:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

This is a useful technique when debugging your programs.

10. Rules for Naming Variables

★ Most Important

A good variable name should describe the value it represents.

Naming rules

1. A variable name must begin with a letter (`A-Z` , `a-z`) or an underscore (`_`).
2. It can contain letters, digits, and underscores.
3. It cannot begin with a digit.
4. Variable names are case-sensitive.
5. Python keywords, such as `if` , `else` , and `class` , cannot be used as ordinary variable names.

Valid names	Invalid names
`name`	`2name`
`first_name`	`first-name`
`age2`	`first name`
`_count`	`class`
`student_age`	`total$price`

Case sensitivity

```
age = 20
Age = 25

print(age)
print(Age)
```

Output:

```
20
25
```

Python treats `age` and `Age` as two different variable names.

Recommended naming style

Python commonly uses `snake_case` , where words are separated by underscores.

```
first_name = "Ashwani"
student_age = 20
total_price = 500
is_logged_in = True
```

Choose descriptive names instead of unclear names such as `x` , `a1` , or `data` , unless those names make sense in the context.

11. Common Mistakes to Avoid

Frequently Asked

Mistake 1: Putting quotes around a number unintentionally

```
age = "20"
```

This creates a string, not an integer.

Correct when storing an integer:

```
age = 20
```

Mistake 2: Using a variable name inside quotes

```
name = "Ashwani"
print("name")
```

Output: **name**

To display the variable's value:

```
print(name)
```

Output: **Ashwani**

Mistake 3: Writing Boolean values in lowercase

```
is_student = true
```

Correct:

```
is_student = True
```

Mistake 4: Forgetting the `f` in an f-string

```
name = "Ashwani"
print("Hello, {name}")
```

Output: **`Hello, {name}`**

Correct:

```
print(f"Hello, {name}")
```

Output: **`Hello, Ashwani`**

Mistake 5: Trying to add a string directly to an integer

```
age = 20
# print("Age: " + age)  # Raises TypeError
```

Use an f-string instead:

```
print(f"Age: {age}")
```

Output: **`Age: 20`**

12. Practice Questions

Try solving these without looking at the solutions first.

Question 1: Identify the data types

Identify the type of each value.

```
a = "Python"
b = 100
c = 25.75
d = False
```

Question 2: Predict the output

```
name = "Ashwani"
age = 20

print(f"My name is {name} and I am {age} years old.")
```

Question 3: Find the error

```
is_online = true
print(is_online)
```

What should be corrected?

Question 4: Choose the correct type

Which data type is most suitable for each value?

- A person's name
- The number of books in a bag
- The price of a product
- Whether a user is logged in

Question 5: Write your own program

Create four variables:

- **`favorite_language`** – a string
- **`number_of_projects`** – an integer
- **`study_hours`** – a float
- **`is_learning_python`** – a Boolean

Print all four values, using f-strings where appropriate.

Solutions

1. Data types: `a` is `str`, `b` is `int`, `c` is `float`, and `d` is `bool`.
2. Output:

```
My name is Ashwani and I am 20 years old.
```

3. Correction: Replace `true` with `True`.
4. Suitable types: String, integer, float, and Boolean, respectively.
5. Sample solution:

```
favorite_language = "Python"
number_of_projects = 3
study_hours = 2.5
is_learning_python = True

print(f"Favorite language: {favorite_language}")
print(f"Number of projects: {number_of_projects}")
print(f"Study hours: {study_hours}")
print(f"Learning Python: {is_learning_python}")
```

13. Quick Revision Sheet

Shortcut Summary

Concept	Remember
Variable	A name that refers to a value
Assignment	`name = "Ashwani"`
String	Text enclosed in quotes
Integer	Whole number, such as `25`
Float	Number with a fractional part, such as `25.5`
Boolean	`True` or `False`
Print a variable	`print(name)`
Print literal text	`print("name")`
F-string	`f"Hello, {name}"`
Check data type	`type(variable)`
Check a condition	`if is_student:`
Naming convention	`student_name`, `total_price`
Case sensitivity	`age` and `Age` are different names

The four data types at a glance

```
name = "Ashwani"      # str
age = 20              # int
gpa = 3.2             # float
is_student = True     # bool
```

Final Takeaway

★ Exam & Interview Core

A variable lets you work with a value through a meaningful name. The most important beginner skill is knowing the difference between **text (`str`), whole numbers (`int`), decimal numbers (`float`), and truth values (`bool`)**.

Practice creating these four types, displaying them with `print()`, inserting values into f-strings, and using Boolean variables in `if` / `else` statements. These concepts form the foundation for more advanced Python topics such as functions, loops, data structures, and object-oriented programming.



